TCLNET: LEARNING TO LOCATE TYPHOON CENTER USING DEEP NEURAL NETWORK (SUPPLEMENTAL MATERIAL)

Chao Tan

istvartan@outlook.com

Table 1. The architecture of TCLNet. Conv means convolutional layer, ConvBlock means convolution block and ResBlock means residual block. **OUTPUT** denotes the amount of output channels in the current layer, and **KERNEL** means the convolutional kernel size. The convolutional kernel moving stride for all layers is 1 except for the layer name ending with S2 which is 2.

LAYER	OUTPUT	KERNEL	LAYER	OUTPUT	KERNEL
Conv-S2	16	7x7	Maxpooling	256	-
ConvBlock	32	1x1	ResBlock	256	3x3
ResBlock	32	3x3	ResBlock	256	3x3
MaxPooling	32	-	UpSample	256	-
ResBlock	32	3x3	ResBlock	128	3x3
Conv	64	1x1	UpSample	128	-
ResBlock	64	3x3	ResBlock	64	3x3
ResBlock	128	3x3	UpSample	64	-
Maxpooling	128	-	ResBlock	64	3x3
ResBlock	256	3x3	ConvBlock	64	1x1
Maxpooling	256	-	Conv	1	1x1
ResBlock	256	3x3	-	-	-

1. DETAILED NETWORK STRUCTURE

The overall network of TCLNet is a standard fully convolutional encoding-decoding structure, and uses max-pooling and upsampling to adjust the range of the receptive fields. The core component of TCLNet is ResBlock, and the reason for using residual network [1] is that it can obtain fewer parameters and better performance by learning residual functions with reference to the layer inputs instead of learning unreferenced functions, opposed to traditional convolutional networks. Specifically, as shown in Figure 1, each residual block compresses the input feature maps through 1×1 convolution layer to reduce the amount of calculation and parameters for the subsequent convolution operations firstly, and then two ConvBlocks and one 1×1 convolution layer are applied for residual features extraction and channel expansion respectively. The final output of residual block is the addition of input and residual feature maps connected by internal skip connection. The detailed structure of TCLNet is shown in Table 1. It can be seen that TCLNet follow the simplest end-to-end network architecture and does not use other strategies like skip connection between encoding and decoding layers or deep supervision.

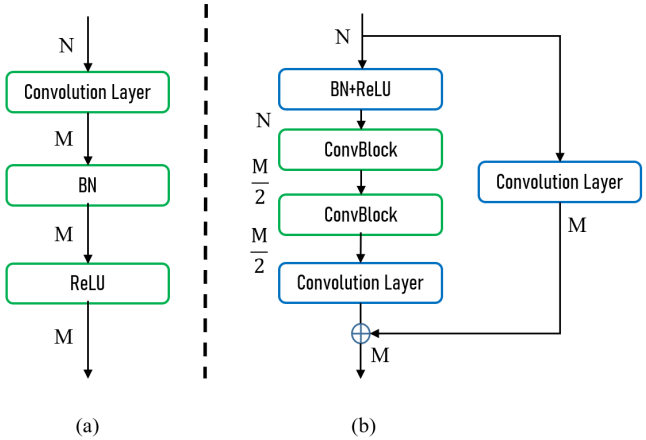

Fig. 1. The core components of convolution block (a) and residual block (b) in TCLNet. BN indicates batch normalization [2] layer and ReLU is Rectified Linear Unit [3]. N and M are the number of input and output feature map channels respectively.

2. TRAINING STRATEGIES

We use the Adam solver [4] with a basic learning rate of 0.001 and the first and second momentum values are 0.5 and 0.999 respectively to optimize our network. To be fair, the size of input infrared cloud imagery is 512×512 for all the experiments, and the size of output typhoon center heatmap is 128×128. In terms of data argumentation, we scale the images from dataset to 574×574 and then randomly cropped to 512×512 with random flips. Unless explicitly specified, we adopt mean square loss (MSE) for all experiments and set the scaling factor α and standard deviation σ to 0.25 and 15 respectively. We train TCLNet in a total of 65 epochs with a mini-batch size of 4, and we reduce the learning rate from 10^{-3} to 10^{-4} after 30 epochs.

When using TCL+ as the loss function during training, in order to make the network converge faster, we use the mean square error (MSE) loss to train the network at the first 50 epochs and switch to TCL+ loss after 50 epochs to continue training until the end. Lastly, we built our model on PyTorch [5] library and trained the model on a computer with Intel Core i7-8700 CPU and GTX1080Ti GPU.

3. EXPERIMENTS ON DIFFERENT HEATMAP LABELS

During the training process, we also analyze the impact of different standard deviations σ used to generate the ground-truth heatmap on the training result. We conduct experiments on TCLNet with standard deviations ranging from 5 to 30 in interval of 5, and noted that except for the different standard deviations used to generate heatmap labels, the other training parameters are all the same settings. The result of six experiments are shown in Table 2. It can be seen that too small standard deviations will lead to too steep probability value distribution of heatmap, which results in performance degradation due to too strict restrictions on the output of the model. Conversely, a too large standard deviation will increase the supervision error during the training process since the probability value distribution of heatmap is too flat. As can be seen from Table 2 that the best effect can be achieved when the standard deviation σ is set to 15.

Table 2. Experimental results of different standard deviations used to generate heatmap labels. STD. means the standard deviation σ.

STD.	MLE-A	MLE-E	MLE-N
5	5.1253±0.0741	3.4311±0.1437	8.2564±0.3290
10	4.6345±0.0417	3.0714±0.1158	7.5235±0.1912
15	**4.5137**±0.0846	**2.8934**±0.0620	**7.5083**±0.1684
20	4.7810±0.1145	3.0845±0.0616	7.9165±0.3020
25	4.9612±0.0391	3.2148±0.0640	8.1890±0.1735
30	5.7168±0.1376	4.2104±0.2082	8.5009±0.2371

4. REFERENCES

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016. 1

[2] Sergey Ioffe and Christian Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015. 1

[3] Xavier Glorot, Antoine Bordes, and Yoshua Bengio, "Deep sparse rectifier neural networks," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011, pp. 315–323. 1

[4] Diederik P Kingma and Jimmy Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014. 1

[5] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer, "Automatic differentiation in pytorch," 2017. 1